

Kernels

2026 Semester 1 SOFTENG 370: Operating Systems

Talia Xu

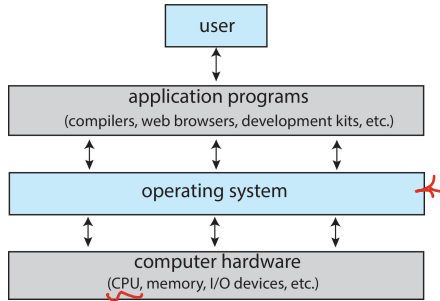
Lecture 4

2.0.0

Based on original slides by Professor Jon Eyolfson.

This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/)

Abstract View of Components of Computer



“The one program running at all times on the computer” is the kernel, the core component of the operating system.

Kernel Operates in a Privileged Kernel Mode

The **kernel** is a piece of software that controls hardware. It is a compiled program (mostly C and Assembly code) written by developers.

The kernel operates in the **kernel mode**. When the CPU is in this state, it is allowed to execute *any* instruction and access *any* memory address.

User programs operate in **user mode**, a restricted state where the CPU blocks direct access to hardware and system memory.

You Can Think of the Kernel as a Long Running Process

Writing kernel code is more like writing library code (there's no main)

The kernel lets you load code (called modules)

Your code executes on-demand

e.g. when it's loaded manually, new hardware, or accessing a certain file

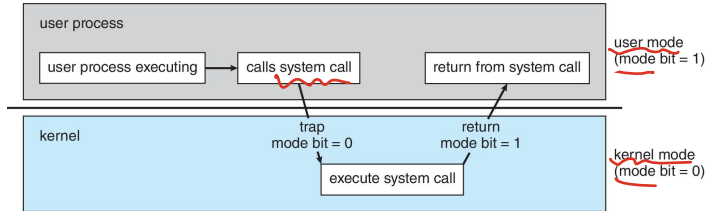
If you write a kernel module, you can execute privileged instructions and access any kernel data, so you could do anything

Dual-mode Operation

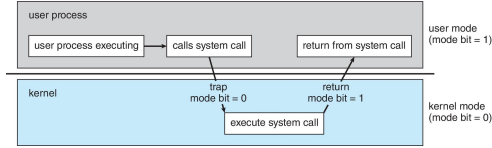
user mode
kernel.

Dual-mode operation allows OS to protect itself and other system components.

To perform privileged tasks, a user program must cross this boundary by making a system call to the kernel.



Dual-mode Operation



```
int main(void) {  
    write(1, "1", 1); // printf("1");  
    return 0;  
}
```

↑ standard function.
← system call. = function.

How does OS know this is a system call?

How does OS know which system call this is? (Recall we (users) have no access to kernel memory).

API.

ABI (App. Binary Interface).

1. during execution. kernel checks?
2. compile time?

↳ ISA. ABI.

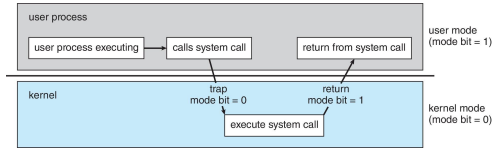
Argument?

register.

↓
a pointer to user space memory

write → system call IO.
(code seg. virtual memory).

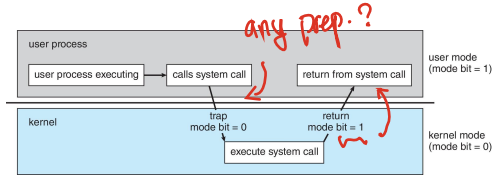
Dual-mode Operation



```
int main(void) {  
    write(1, "1", 1); // printf("1");  
    return 0;  
}
```

How do we pass arguments into the kernel? In a normal function call we often use the stack, but here we are crossing into the kernel.

Dual-mode Operation



PC. →

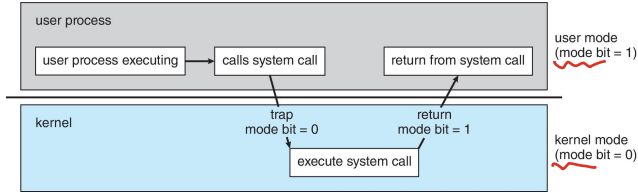
```
int main(void) {  
    write(1, "1", 1); // printf("1");  
    return 0;  
}
```

Do we need any preparation before entering the kernel?

How do user code and kernel code know where to put the return value?

compiled time.

Dual-mode Operation



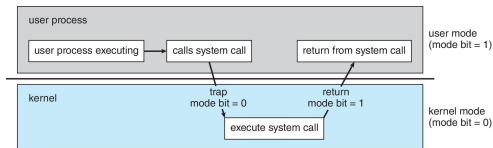
Who changes the mode bit? The user program or the kernel?

user ? $\leftarrow$ No. unsafe.

kernel ? $\leftarrow$ No. software.

Hardware. $\leftarrow$ CPU. (enforced by hardware)
isolation & security.

Dual-Mode Operation: Before, During, and After a System Call



Before

ISA: entry instruction

ABI: syscall no., args, re-
turn value

Compiler / libc: generate
code to follow rules

During ↵

User: place syscall no. and
args

User: execute syscall

CPU: switch to kernel
mode

Kernel: validate and per-
form operation

After

Kernel: place return value *reg.*

CPU: restore user mode

User: continue with result

There's 3 Major ISAs in Use Today

ISA stands for the *instruction set architecture*

It's the machine code, or numbers the CPU understands

- x86-64 (aka amd64): for desktops, non-Apple laptops, servers
- aarch64 (aka arm64): for phones, tablets, Apple laptops
- riscv (aka rv64gc): open-source implementation, similar to ARM

We'll touch on all of them in this course

API Tells You What and ABI Tells You How

Application Programming Interface (API) abstracts the details and describes the arguments and return value of a function

e.g. A function takes 2 integer arguments

Application Binary Interface (ABI) specifies the details, specifically how to pass arguments and where the return value is

e.g. The same function using the C calling convention
(arguments on the stack)

Dual-Mode Operation: The Mode Bit

A **mode bit** is provided by the hardware to distinguish when the system is running user code versus kernel code.

When a user program is running → mode bit is set to “user”.

When kernel code is executing → mode bit is set to “kernel”.

Changing the mode bit is a **privileged instruction**, which can *only* be executed by the hardware if it is already in kernel mode.

A **system call** safely triggers the hardware to switch the mode to kernel, and returning from that call automatically resets it back to user mode.

System Calls Make Requests to the Operating System

We can represent system calls like regular C functions

Here are two system calls we need for a basic "Hello world" program:

→ `ssize_t write(int fd, const void *buf, size_t count);`

Description: writes bytes from a byte array to a file descriptor

fd - the file descriptor

buf - the address of the start of the byte array (called a buffer)

count - how many bytes to write from the buffer

→ `void exit_group(int status);`

Description: exits the current process and sets an exit status code

status - the exit status code (0-255)

return 0: normal.
-1: .

Our Next Abstraction is a File Descriptor

Since our processes are independent, we need an explicit way to transfer data

IPC: inter-process communication is transferring data between two processes

→ index into a resource table.

⇒ **File descriptor**: a resource that users may either read bytes from or write bytes to
(identified by an index stored in a process) →

A file descriptor could represent a file, or your terminal, screen,
serial port.

A Hypothetical "Hello world" Program

By convention there's some expected file descriptors:

index - [

- 0 - standard input (read)
- 1 - standard output (write)
- 2 - standard error (write)

The most basic "Hello world" program would start executing the following:

```
void _start(void) {  
    write(1, "Hello world\n", 12);  
    exit_group(0);  
}
```

Handwritten annotations:

- # of bytes.* (pointing to 12)
- printf("Hello World\n");* (written next to the write function)
- // return 0;* (written next to exit_group(0))
- * buf.* (pointing to "Hello world\n")
- address "H".* (pointing to the first character of the string)
- fd.* (pointing to 1)
- stdout.* (pointing to 1)

System Calls Are Traceable

We can trace all the system calls a process makes on Linux using the command:

```
strace <PROGRAM>
```

We can see all the system calls our “Hello world” program makes:

```
execve("./hello_world", [ "./hello_world" ], 0x7ffd0489de40 /* 46 vars */) = 0
write(1, "Hello world\n", 12)          = 12
exit_group(0)                          = ?
+++ exited with 0 +++
```

Now, let's really see what C does...

System Calls for “Hello world” in C, Finding Standard Library

```
execve("./hello_world_c", ["/hello_world_c"], 0x7ffcb3444f60 /* 46 vars */) = 0
brk(NULL)                                = 0x5636ab9ea000
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=149337, ...}) = 0
mmap(NULL, 149337, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f4d43846000
close(3)                                 = 0
openat(AT_FDCWD, "/usr/lib/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0000C"..., 832) = 832
lseek(3, 792, SEEK_SET)                   = 792
read(3, "\4\0\0\0\24\0\0\0\3\0\0\0GNU\0\201\336\t\36\251c\324"..., 68) = 68
fstat(3, {st_mode=S_IFREG|0755, st_size=2136840, ...}) = 0
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f4d43844000
lseek(3, 792, SEEK_SET)                   = 792
read(3, "\4\0\0\0\24\0\0\0\3\0\0\0GNU\0\201\336\t\36\251c\324"..., 68) = 68
lseek(3, 864, SEEK_SET)                   = 864
read(3, "\4\0\0\0\20\0\0\0\5\0\0\0GNU\0\2\0\0\300\4\0\0\0\3\0\0", 32) = 32
```

System Calls for “Hello world” in C, Loading Standard Library

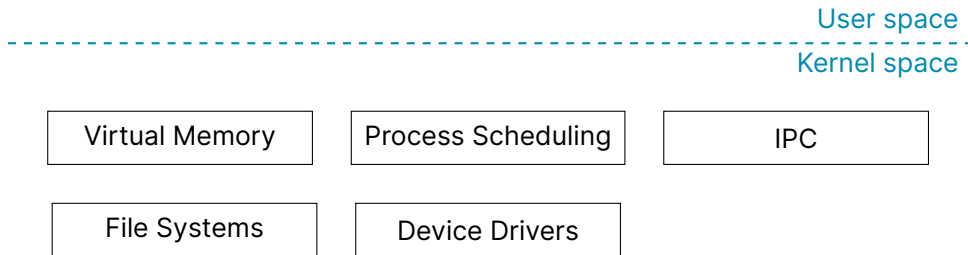
```
mmap(NULL, 1848896, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f4d43680000
mprotect(0x7f4d436a2000, 1671168, PROT_NONE) = 0
mmap(0x7f4d436a2000, 1355776, PROT_READ|PROT_EXEC,
      MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x22000) = 0x7f4d436a2000
mmap(0x7f4d437ed000, 311296, PROT_READ,
      MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x16d000) = 0x7f4d437ed000
mmap(0x7f4d4383a000, 24576, PROT_READ|PROT_WRITE,
      MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1b9000) = 0x7f4d4383a000
mmap(0x7f4d43840000, 13888, PROT_READ|PROT_WRITE,
      MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7f4d43840000
close(3) = 0
arch_prctl(ARCH_SET_FS, 0x7f4d43845500) = 0
mprotect(0x7f4d4383a000, 16384, PROT_READ) = 0
mprotect(0x5636a9abd000, 4096, PROT_READ) = 0
mprotect(0x7f4d43894000, 4096, PROT_READ) = 0
munmap(0x7f4d43846000, 149337) = 0
fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0x1), ...}) = 0
```

System Calls for “Hello world” in C, Setting Up Heap and Printing

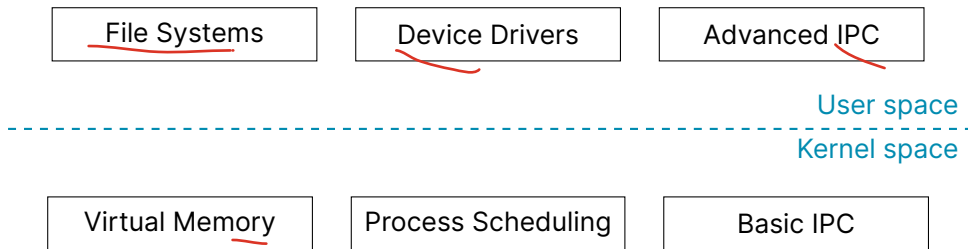
<code>brk(NULL)</code>	<code>= 0x5636ab9ea000</code>
<code>brk(0x5636aba0b000)</code>	<code>= 0x5636aba0b000</code>
<code>write(1, "Hello world\n", 12)</code>	<code>= 12</code>
<code>exit_group(0)</code>	<code>= ?</code>
<code>+++ exited with 0 +++</code>	

The C version of “Hello world” ends with the exact same system calls we made

A Monolithic Kernel Runs Operating System Services in Kernel Mode



A Microkernel Runs the Minimum Amount of Services in Kernel Mode



Other Types of Kernels

“Hybrid” kernels are between monolithic and microkernels

- Emulation services to user mode (Windows)

- Device drivers to user mode (macOS)

Nanokernels and picokernels

- Move even more into user mode than traditional microkernels

There's different architectural lines you can draw with different trade-offs

Kernel Interfaces Operate Between CPU Mode Boundaries

Main takeaways from the lecture:

- The kernel is the part of the OS that interacts with hardware (it runs in kernel mode)
- System calls are the interface between user and kernel mode
 - Every program must use this interface!
- Different kernel architectures shift how much code runs in kernel mode

Review Question: What is the output?

```
#include <stdio.h>
#include <unistd.h>

int main(void) {
    printf("1");
    sleep(1);
    write(1, "2", 1);
    return 0;
}
```